

UNIVERSIDADE DO VALE DO RIO DOS SINOS – UNISINOS
UNIDADE ACADÊMICA DE GRADUAÇÃO
GRADUAÇÃO EM ENGENHARIA

BRUNO RODRIGUES FERNANDES DOS ANJOS
ERIK CRUZ MORBACH
LUCAS ESCOPELLI

AGRO HASHING

SÃO LEOPOLDO

2025

BRUNO RODRIGUES FERNANDES DOS ANJOS
ERIK CRUZ MORBACH
LUCAS ESCOPELLI

AGRO HASHING

Trabalho da atividade acadêmica de Algoritmos e Programação: Estruturas Avançadas de Dados apresentado como avaliação parcial do grau A, na Universidade do Vale do Rio dos Sinos (UNISINOS).

Orientador: Prof. Márcio Garcia Martins

SÃO LEOPOLDO
2025

1 PRINCIPAIS PONTOS DA IMPLEMENTAÇÃO

O projeto AgroHashing consiste em um sistema de gerenciamento de sensores agrícolas desenvolvido em C++, utilizando estruturas de dados baseadas em tabelas hash. A arquitetura do sistema foi projetada seguindo princípios de orientação a objetos, com foco em extensibilidade e reutilização de código através do uso de templates e herança.

No núcleo da implementação encontra-se a interface abstrata `HashTable<T>`, que define o contrato fundamental para todas as implementações de tabelas hash. Esta interface especifica os métodos essenciais: `add()` para inserção de elementos, `remove()` para remoção, `search()` para busca, `listAll()` para listagem completa dos elementos armazenados, e `getCollisions()` para obter estatísticas de colisões. Essa abstração permite que diferentes estratégias de implementação sejam utilizadas de forma intercambiável.

Complementando a arquitetura, a interface **Hashable<T>** permite que qualquer objeto defina sua própria função de dispersão através do método virtual `hash()`. Esta abordagem proporciona flexibilidade ao sistema, pois cada tipo de dado pode implementar uma função hash otimizada para suas características específicas.

A entidade de domínio principal é a classe `Sensor`, que representa os sensores agrícolas do sistema. Cada sensor possui quatro atributos fundamentais: um identificador numérico único (`id`), o tipo do sensor (como temperatura, umidade ou pH), sua localização no campo, e a última leitura registrada. A classe `Sensor` implementa a interface `Hashable`, fornecendo assim sua própria função de dispersão.

O projeto oferece duas implementações distintas de tabelas hash: **LinkedHashTable**, que utiliza a estratégia de encadeamento para tratamento de colisões, e **OpenHashTable**, que emprega endereçamento aberto com sondagem linear. Essas duas abordagens representam as principais famílias de técnicas para resolução de colisões em tabelas hash.

2 FUNÇÃO HASH

A função hash implementada na classe Sensor utiliza uma técnica de **multiplicação por número primo**, uma abordagem clássica na literatura de estruturas de dados que visa melhorar a dispersão dos valores hash. O número primo escolhido foi 1000000007, um primo grande frequentemente utilizado em competições de programação e sistemas que requerem boa distribuição de valores.

A implementação em código é a seguinte:

```
#define PRIMO 1000000007LL

unsigned long long int Sensor::hash() const {
    return this->id * PRIMO;
}
```

O índice final na tabela é então calculado aplicando a operação módulo sobre o valor hash retornado, utilizando o tamanho da tabela como divisor: $\text{índice} = \text{hash}() \% \text{tamanho_tabela}$. Esta operação garante que o índice resultante esteja sempre dentro dos limites válidos do array subjacente.

A multiplicação por um número primo grande oferece várias vantagens em relação à abordagem de identidade simples. Primeiramente, ela quebra padrões sequenciais nos IDs de entrada, distribuindo os valores de forma mais uniforme pela tabela. Além disso, a propriedade matemática dos números primos garante que, quando combinada com a operação módulo por um tamanho de tabela coprimo, a função produza uma distribuição mais equilibrada. Esta técnica é especialmente eficaz quando os IDs de entrada são sequenciais ou possuem padrões previsíveis, situação comum em sistemas de sensores onde os identificadores são atribuídos incrementalmente.

3 ESTRATÉGIAS DE COLISÃO

3.1 ENCADEAMENTO (CHAINING)

A primeira estratégia implementada, presente na classe **LinkedHashTable**, utiliza o método de encadeamento para tratamento de colisões. Nesta abordagem, cada posição da tabela hash não armazena diretamente um único elemento, mas sim uma lista encadeada `std::list<std::shared_ptr<T>>` que pode conter múltiplos elementos.

Quando dois ou mais elementos são mapeados para o mesmo índice através da função hash, todos eles são armazenados na lista correspondente àquela posição. A inserção de novos elementos é realizada no início da lista através do método `push_front()`, garantindo complexidade $O(1)$ para esta operação. A busca, por sua vez, requer percorrer sequencialmente a lista até encontrar o elemento desejado, resultando em complexidade $O(n)$ no pior caso, onde n é o número de elementos na lista específica.

O sistema contabiliza as colisões incrementando um contador sempre que uma inserção é realizada em uma posição da tabela que já contém elementos. Esta métrica permite avaliar a qualidade da função hash e o dimensionamento adequado da tabela.

3.2 ENCADEAMENTO (CHAINING)

A segunda estratégia, implementada na classe **OpenHashTable**, utiliza endereçamento aberto com sondagem linear. Diferentemente do encadeamento, esta técnica armazena todos os elementos diretamente nas posições da tabela, sem utilizar estruturas de dados auxiliares.

Cada slot da tabela possui um estado associado que pode ser: **EMPTY** (vazio e nunca utilizado), **OCCUPIED** (contém um elemento válido) ou **DELETED** (elemento foi removido, mas a posição já foi utilizada). Esta distinção de estados é fundamental para o correto funcionamento das operações de busca e remoção.

Quando ocorre uma colisão durante a inserção, o algoritmo de sondagem linear procura a próxima posição disponível de forma sequencial, calculando o novo índice como $(\text{hash_inicial} + \text{passo}) \% \text{tamanho}$, onde o passo é incrementado a cada tentativa. Este processo continua até encontrar uma posição vazia ou marcada como deletada.

A remoção de elementos utiliza a técnica de **lazy deletion** (remoção preguiçosa),

marcando a posição como **DELETED** ao invés de efetivamente remover o elemento. Esta abordagem é necessária para manter a integridade das buscas, pois uma posição verdadeiramente vazia indica o fim de uma cadeia de sondagem, enquanto uma posição marcada como deletada indica que a busca deve continuar.

4 RESULTADO COMPORATIVO

Para avaliar o desempenho das duas implementações, foram realizados testes de benchmark com diferentes volumes de dados. Os testes mediram o tempo de inserção e busca de elementos, além de contabilizar o número de colisões ocorridas. A seguir são apresentados os resultados obtidos.

Tabela 1 – Benchmark com 1000 sensores - Encadeamento

Métrica	Tempo de Inserção	Tempo de Busca	Colisões
Encadeamento (101 buckets)	0.81 ms	0.812 ms	899
Encadeamento (501 buckets)	0.864 ms	0.791 ms	499
Encadeamento (1001 buckets)	0.809 ms	0.644 ms	119
Encadeamento (10001 buckets)	0.869 ms	0.72 ms	0

Tabela 2 – Benchmark com 1000 sensores - Endereçamento Aberto com Sondagem Linear

Métrica	Tempo de Inserção	Tempo de Busca	Colisões
Endereçamento (2003 slots)	0.778 ms	0.7 ms	432
Endereçamento (5003 slots)	0.745 ms	0.612 ms	0
Endereçamento (10003 slots)	0.792 ms	0.696 ms	0

Tabela 3 – Benchmark com 5000 sensores - Encadeamento

Métrica	Tempo de Inserção	Tempo de Busca	Colisões
Encadeamento(101 buckets)	4.248 ms	4.291 ms	4899
Encadeamento(501 buckets)	4.521 ms	3.677 ms	4499
Encadeamento(1001 buckets)	3.868 ms	3.467 ms	3999
Encadeamento(10001 buckets)	3.779 ms	3.77 ms	726

Tabela 4 – Benchmark com 1000 sensores - Endereçamento Aberto com Sondagem Linear

Métrica	Tempo de Inserção	Tempo de Busca	Colisões
Endereçamento (5003 slots)	3.766 ms	3.5 ms	1158
Endereçamento (10003 slots)	3.205 ms	3.343 ms	126

A análise dos resultados revela padrões importantes sobre o comportamento das duas implementações em diferentes configurações. Com a nova função hash baseada em multiplicação por número primo, observa-se uma melhoria significativa na distribuição dos elementos, resultando em tempos de operação mais consistentes e previsíveis.

Em relação às colisões no cenário com 1000 sensores, o encadeamento apresentou uma redução gradual conforme o tamanho da tabela aumenta: de 899 colisões com 101 buckets para zero colisões com 10001 buckets. O endereçamento aberto com 2003 slots registrou 432 colisões (fator de carga 50%), mas eliminou completamente as colisões com tabelas de 5003 slots ou maiores. Este comportamento demonstra que, quando adequadamente dimensionada, a função hash com multiplicação por primo distribui os elementos de forma eficiente.

Quanto aos tempos de operação com 1000 sensores, ambas as estratégias apresentaram desempenho similar, com tempos de inserção entre 0.74 ms e 0.87 ms, e tempos de busca entre 0.61 ms e 0.81 ms. O endereçamento aberto com 5003 slots obteve o melhor tempo de busca (0.612 ms), enquanto o encadeamento com 1001 buckets apresentou o melhor equilíbrio entre colisões (119) e tempo de busca (0.644 ms).

No cenário com 5000 sensores, as diferenças entre as estratégias tornam-se mais evidentes. O encadeamento com 101 buckets apresentou 4899 colisões (fator de carga 49.5 elementos/bucket), resultando em tempos de busca mais elevados (4.291 ms). Mesmo com 10001 buckets, o encadeamento ainda registrou 726 colisões, indicando que a distribuição não é perfeitamente uniforme para este volume de dados. O endereçamento aberto com 5003 slots (fator de carga 99.9%) apresentou 1158 colisões, mas manteve tempos de busca competitivos (3.205 ms). Com 10003 slots (fator de carga 50%), as colisões reduziram para 126 e o tempo de inserção foi o mais baixo (3.5 ms).

Um aspecto importante a destacar é que o endereçamento aberto demonstrou maior sensibilidade ao fator de carga. Com fator próximo a 100% (5003 slots para 5000 elementos), houve degradação significativa nas colisões. Já o encadeamento mostrou-se mais resiliente a altos fatores de carga, mantendo operações funcionais mesmo com 50 elementos por bucket, embora com penalidade nos tempos de busca.

5 CONCLUSÃO

A implementação do projeto AgroHashing demonstrou na prática as características teóricas das duas principais estratégias de tratamento de colisões em tabelas hash. A atualização da função hash para utilizar multiplicação por número primo ($10 + 7$) resultou em melhor distribuição dos elementos e desempenho mais previsível em ambas as implementações.

O encadeamento mostrou-se uma opção mais flexível e tolerante a variações no fator de carga, adequada para cenários onde o número de elementos é imprevisível ou pode variar significativamente ao longo do tempo. Sua principal vantagem é não requerer redimensionamento da tabela quando o número de elementos aumenta, embora o desempenho de busca possa degradar com fatores de carga muito elevados. Os testes mostraram que mesmo com 50 elementos por bucket, o sistema permanece funcional.

O endereçamento aberto apresentou os melhores tempos de busca quando adequadamente dimensionado (fator de carga 50%), beneficiando-se da localidade de cache proporcionada pelo uso de arrays contíguos. Entretanto, esta abordagem requer dimensionamento cuidadoso da tabela para evitar degradação de performance, sendo recomendado manter o fator de carga abaixo de 70%. Com fator próximo a 100%, o número de colisões aumenta significativamente.

Em suma, a escolha entre encadeamento e endereçamento aberto deve ser baseada nas características específicas da aplicação. Para sistemas com volume de dados variável e imprevisível, o encadeamento oferece maior flexibilidade. Para aplicações onde o número de elementos é conhecido e as operações de busca são prioritárias, o endereçamento aberto com dimensionamento adequado proporciona melhor desempenho. A função hash com multiplicação por primo mostrou-se eficaz em ambos os cenários, contribuindo para uma distribuição mais uniforme dos elementos

